

Yazılım Geliştirme Lab II

Egemen Moustafa Chaita
Bilişim Sistemleri Mühendisiği
mustafahaitaa@gmail.com

Umutcan Bozyiğit
Bilişim Sistemleri Mühendisiği
umutcanbozyigit34012@gmail.com

Ahmet Faruk Çatlar
Bilişim Sistemleri Mühendisiği
ahmetfaruk411@gmail.com

Drone projesi, birden fazla insansız hava aracının (drone) teslimat rotalarını optimize etmeyi amaçlayan bir simülasyon ve algoritma çalışmasıdır. Bu proje kapsamında, belirli ağırlık kapasiteleri ve batarya limitleri olan dronelerin, çeşitli teslimat noktalarına paket ulaştırması planlanır.

I. GİRİŞ

Her teslimatın bir **öncelik değeri** ve belirli bir **zaman penceresi** (örneğin 09:00-10:00 arası) bulunmaktadır. Ayrıca harita üzerinde drone'ların **uçuş yapmasının yasak olduğu bölgeler (No-Fly Zone)** tanımlanmıştır. Projenin amacı, **mümkün olduğunca çok teslimatı, kurallara uyarak ve verimli şekilde tamamlamak**; yani teslimat sayısını maksimize ederken, mesafe (enerji tüketimi) ve kural ihlallerini minimize etmektir. Bu amaç, kodda tanımlanan **fitness (uygunluk) fonksiyonunda** açıkça görülmektedir: her başarılı teslimat için +50 puan verilirken, her kural ihlali için -1000 ve zaman pencere ihlali için -500 cezalar uygulanmakta (ayrıca harcanan enerjiye küçük bir ceza eklenmekte) GitHub. Bu sayede algoritma, en yüksek puanı almak üzere **maksimum teslimat, minimum ihlal/enerji** dengesini gözetir.

Proje kapsamı, **farklı senaryoları** deneyerek algoritmayı test etmeyi içerir. Kod içerisinde küçük ölçekli ve büyük ölçekli birden fazla senaryo tanımlanmıştır. Örneğin “Temel Senaryo” dışında “**Büyük Senaryo**”, “**Çok Büyük Senaryo**” gibi farklı veri setleri hazırlanmıştır; bunlar drone ve teslimat sayıları arttıkça popülasyon boyutu ve jenerasyon sayısını da yükselterek algoritmayı zorlar GitHub. Ayrıca **rastgele senaryo üretme** özelliği de bulunmaktadır – istenirse parametrelerle rastgele konumlarda teslimat noktaları ve no-fly zone bölgeleri oluşturularak yeni bir senaryo otomatik yaratılır. Bu kapsam dahilinde proje; **senaryo verisi oluşturma**, algoritmaları çalıştırma, sonuçları analiz etme ve görselleştirmeyi içeren uçtan uca bir çözüm sunar.

II. YAZILIM MIMARISI

Projenin yazılım mimarisi temel olarak **üç ana bileşene** ayrılabilir: **Veri Katmanı**, **Çekirdek İşlem Katmanı** ve **Çıktı Katmanı**. Veri katmanı, sistemin girdi aldığı kısımdır ve senaryo verilerinin oluşturulması veya yüklenmesini kapsar. Çekirdek katmanda asıl **iş mantığı ve algoritmalar** yer alır – teslimat atama problemini çözmek için genetik algoritma ve opsiyonel olarak en kısa yol araması (A*) bu bölümde çalışır. Çıktı katmanı ise elde edilen çözümü analiz eden ve kullanıcıya sunan kısımdır; konsola metriklerin yazdırılması ve grafiksel arayüzde rotaların çizilmesi bu bölümde gerçekleşir.

Veri akışı şu şekilde özetlenebilir: Önce sistem, ya hazır tanımlanmış dosyalardan ya da rastgele üretimle **teslimat**

senaryosu verisini alır (drone listesi, teslimat listesi, no-fly zone listesi). Ardından çekirdek katmanda bu veriler işlenir: Drone ve teslimat bilgileri ile kısıtlar, genetik algoritmaya girdi olur; eğer rota planlamasında engelli bölgeler aşılması gerekiyorsa A* algoritması devreye girebilir. Son olarak bulunan çözüm (her bir drone için teslimat sıraları) değerlendirilir, **başarı yüzdesi, ihlal sayıları ve fitness puanı** hesaplanarak konsolda raporlanır ve **rotalar görsel olarak çizilir**. Bu mimariyi özetleyen genel veri akışı diyagramı, Bölüm 8’de sunulmuştur.

Uygulama, **modüler bir yapıdadır**: Her işlevsellik ayrı bir Python modülü (dosyası) içinde tanımlanmıştır. Bu modüller arasındaki ilişkiyi main.py organize eder. Örneğin, main.py içinde önce data_loader ile dosyalardan veri okunur, sonra genetik algoritma çalıştırılır ve en sonda visualize modülü ile çizim yapılır GitHub GitHub. Bu katmanlı yaklaşım, kodun anlaşılmasını ve bakımını kolaylaştırmaktadır. Çekirdek algoritma katmanı doğrudan veri katmanından nesneleri alır ve çıktı katmanına sonuç nesneleri (çözüm) üretir. Veri ve işlem katmanları arasında gevşek bir bağ vardır; örneğin, girdi verisi ister dosyadan ister rastgele oluşturulsun, **aynı veri yapısına dönüştürüldüğü** için algoritma kısmı hep aynı şekilde çalışır. Benzer şekilde, çıktı katmanı da algoritmadan aldığı çözümü standart biçimde ele alır. Bu sayede mimarinin parçaları birbirinden bağımsız geliştirilebilir.

III. KULLANILAN TEKNOLOJILER

Proje %100 Python dilinde yazılmıştır github.com. Python’un yüksek seviye dil özellikleri ve zengin kütüphane desteği, hem hızlı prototipleme hem de bilimsel hesaplamalar için uygun bir altyapı sağlamıştır. Kod Python 3’ün **dataclasses** ve **typing** gibi modern modüllerini kullanmaktadır (örneğin Drone ve diğer veri sınıfları @dataclass ile tanımlanmış, tip ipuçlarıyla okunabilirlik artırılmıştır). Çekirdek algoritmalar saf Python ile yazılmış, herhangi bir özel framework ya da dış API kullanılmamıştır.

Üçüncü parti kütüphanelerden en önemlisi **Matplotlib**’dir. **Matplotlib**, sonuçların görselleştirilmesi için kullanılmıştır – özellikle teslimat rotalarını, drone başlangıç konumlarını ve no-fly zone bölgelerini çizmek için matplotlib.pyplot modülü kullanılmaktadır GitHub. Bunun dışında proje kapsamında harici bir servis veya veritabanı yoktur; tüm veriler basit metin dosyalarında saklanmakta ve tüm işlemler yerel olarak Python içinde gerçekleştirilmektedir. Geliştirme ortamı olarak herhangi bir Python IDE’si (PyCharm, VSCode vb.) kullanılabilir; ek bir derleme veya dağıtım aracı kullanılmadığı görülmektedir. Proje kurulumu için tek gereklilik Python 3 ve Matplotlib’in yüklü

olmasıdır. Bu sadelik, projenin kolay çalıştırılabilir olmasını sağlar.

IV. ANA MODÜLLER

Proje kod tabanında birbirinden sorumlu farklı görevleri üstlenen **ana modüller** şunlardır:

- **drone.py:** Projenin temel veri yapılarını tanımlar. Drone, teslimat noktası (DeliveryPoint) ve no-fly zone için **dataclass**'ler burada bulunmaktadır. Örneğin Drone sınıfı her bir drone için **id**, **taşıma kapasitesi (max_weight)**, **batarya**, **hız** ve **başlangıç pozisyonu** gibi alanları içerirGitHub. Bu veri sınıfları, diğer modüller arasında veri alışverişini nesne bazında kolaylaştırır.
- **data_loader.py:** Dış kaynaklı senaryo verilerini sistem içerisine yüklemek için kullanılır. Bu modül, önceden hazırlanmış metin dosyalarından drone listesi, teslimat listesi ve no-fly zone listesi okuyan fonksiyonlar içerir. Örneğin load_drones fonksiyonu, CSV formatlı bir dosyadan her satırı okuyup Drone nesnesine çevirirGitHub. Benzer şekilde load_deliveries ve load_noflyzones fonksiyonları ilgili dosyaları okuyarak teslimat noktalarını ve no-fly zone bölgelerini DeliveryPoint ve NoFlyZone nesnelerine dönüştürürGitHub. Bu sayede dosya formatından bağımsız olarak, sistem içinde tutarlı veri nesneleri elde edilir.
- **data_generator.py:** Test ve simülasyon amaçlı **rastgele senaryo verisi üretmeye** yarayan modüldür. Parametrik olarak verilen drone sayısı, teslimat sayısı ve no-fly zone sayısına göre, her biri rastgele değerlerle doldurulmuş veri listeleri oluşturur. Örneğin generate_random_drones belirli aralıklarda rastgele ağırlık kapasitesi, batarya, hız ve pozisyon atayarak istenen sayıda drone üretir. Teslimatlar için benzer şekilde rastgele konum, ağırlık, öncelik ve 1 saatlik rastgele zaman penceresi oluşturan generate_random_deliveries fonksiyonu vardırGitHub. No-fly zone'lar için ise harita üzerinde rastgele boyutta kare alanlar üreten generate_random_noflyzones bulunmaktadır. Bu modül aynı zamanda ürettiği veriyi dosyalara yazan save_to_file ve komple bir senaryoyu tek seferde oluşturan generate_scenario fonksiyonlarını da içerir. generate_scenario, belirtilen parametrelerle drone, teslimat ve no-fly zone listelerini üretilen ilgili dosyalara kaydeder ve özet bilgisini ekrana yazdırırGitHub (örneğin kaç drone, kaç teslimat noktası yaratıldığı bilgisini verir). Bu modül sayesinde, gerçek veriye ihtiyaç duyulmadan çeşitli senaryolar test edilebilir.
- **graph_utils.py:** Harita üzerindeki **mesafe hesaplamaları**, **geometrik kontroller** ve **grafik modellemeye** ilgili yardımcı fonksiyonları içerir. En temel fonksiyon euclidean_distance, iki nokta arasındaki Öklid uzaklığını hesaplar. Bunun

yanında, verilen iki doğru parçasının kesişip kesişmediğini bulan yardımcı işlemler (ccw ve segments_intersect) tanımlıdır – bunlar no-fly zone'ların sınırları ile drone uçuş hattının çakışmasını kontrol etmek için kullanılır.

Modüldeki önemli bir fonksiyon da edge_crosses_noflyzones fonksiyonudur; bu fonksiyon bir drone'un geçeceği bir hattın herhangi bir no-fly zone poligonunu **kestiğini tespit etmek** için, hattın her bir no-fly zone kenetiyle kesişimini kontrol ederGitHub. Son olarak build_graph fonksiyonu, tüm teslimat noktalarını ve drone başlangıç noktalarını düğümler olarak alan bir **komşuluk listesi grafi** oluştururGitHub. Bu graf, her bir noktadan (drone veya teslimat) diğer teslimat noktalarına giden kenarlar ve bunların maliyetlerini içerir; maliyet hesaplanırken mesafeye ek olarak teslimat önceliği ve no-fly zone cezaları da dikkate alınır. Ayrıca build_graph fonksiyonu, **drone'ların kapasite ve batarya kısıtlarını** da kenar oluştururken hesaba katar – örneğin bir drone'un bataryası yetmiyorsa ya da paket ağırlığı kapasitesini aşıyorsa ilgili teslimat noktasına bir kenar oluşturulmazGitHub. Bu graf yapısı, A* algoritmasının arka planda kullanılabileceği bir yol haritası sunmak için tasarlanmıştır.

- **astar.py:** Harita grafi üzerinde **en kısa yolu** bulmak için A* algoritmasını uygular. astar fonksiyonu, komşuluk listesi temsilindeki grafi ve başlangıç/hedef düğüm id'lerini alarak, A* arama stratejisiyle iki nokta arasındaki en düşük maliyetli rotayı hesaplar. Kod, klasik A* algoritması adımlarını izler: açık liste (öncelik kuyruğu) kullanarak her iterasyonda en düşük f_score değerine sahip düğümü seçer, komşularına giderken g_score ve f_score değerlerini güncellerGitHub. Hedef düğüme ulaşıldığında, came_from sözlüğünü kullanarak başlangıca kadar yolu geri çıkarır. Fonksiyon bulunacak yolu, düğüm id'lerinden oluşan bir liste olarak döndürürGitHub. A* algoritması, **heuristic** olarak düz çizgi mesafesini (Öklid mesafesi) kullanmıştır (yani $f_score = g_score + heuristic$ formülünde $heuristic = \text{mevcut düğüm} - \text{hedef düz mesafe}$)GitHub. Bu yaklaşım, no-fly zone'ları kenar maliyetlerine ceza ekleyerek hesaba katıyor; dolayısıyla A* bu cezalara rağmen uygun rotayı bulabilir. Ancak mevcut implementasyonda A* fonksiyonu sistem içinde doğrudan çağrılmamış; muhtemelen geliştirme sırasında planlanmış ancak genetik algoritmanın çözümünden sonra ayrı bir detay rotalama adımı olarak düşünülmüş bir bileşen olarak kodda yer alıyor.
- **genetic.py:** Bu modül, asıl optimizasyonu gerçekleştiren **Genetik Algoritma** ve destekleyici fonksiyonlarını içerir. Teslimat görevinin birden fazla drone arasında dağıtılması problemini, genetik algoritmayla yaklaşılarak çözer. Başlıca parçaları şöyle özetlenebilir:

- create_random_chromosome:
Teslimatların drone'lara rastgele dağıtımını yapan bir fonksiyon. Her bir drone bir **kromozom** olarak düşünülebilir; kromozom yapısı {drone_id: [teslimat_id, ...], ...} şeklinde bir sözlüktür. Bu fonksiyon, tüm teslimat id'lerini karıştırıp sırayla drone'ların listelerine paylaştırarak rastgele bir atama oluştururGitHub. Böylece başlangıç popülasyonundaki bireyler için çeşitlilik sağlanır.
- fitness: Verilen bir atama çözümünün **uygunluk puanını** hesaplar. Yukarıda da bahsedildiği gibi, fitness hesaplaması teslimat sayısını artırmaya çalışırken kısıt ihlallerini cezalandıracak şekilde tasarlanmıştırGitHub. Fonksiyon, her bir drone'un rotasını simüle eder ve **kapasite, batarya, no-fly zone ve zaman penceresi** kısıtlarını kontrol eder. Örneğin bir drone'un rotasındaki her teslimat noktası için:
 - Paket ağırlığı drone kapasitesini aşıyorsa o teslimat *yapılamaz* sayılır (kural ihlali olarak sayılıyor).
 - Teslimat noktasına olan mesafe drone'un kalan bataryasından fazlaysa yine teslimat yapılamaz (ihlal).
 - Drone ile teslimat noktası arasındaki doğru çizgi herhangi bir no-fly zone bölgesini kesiyorsa, drone bu rotayı ihlal etmiş kabul edilir; koddaki durumda yine ihlal sayısı artırılıyor, ancak drone *yine de o noktaya gider* (bataryasını harcayıp pozisyonunu oraya taşıyor) – yalnız zaman ilerlemesi durduruluyor, yani ceza olarak drone bekletiliyor.
 - Drone hedefe ulaştığında teslimat zaman penceresine bakılır; eğer belirtilen zaman aralığından erken veya geç gelmişse bu bir *zaman ihlali* olarak sayılır.
 - Yukarıdaki kısıtları aşmayı başaran her teslimat başarılı kabul edilir; drone bataryasından mesafe kadar enerji düşülür, zaman ilerletilir ve başarı sayısı artırılır.

Tüm drone'lar için bu hesaplar yapıldıktan sonra fitness değeri (başarılı teslimat sayısı * 50) - (toplam mesafe * 0.1)

- (kural ihlali * 1000) - (zaman ihlali * 500) formülüyle bulunur. Bu puan ne kadar yüksekse çözüm o kadar iyi demektir.

- create_initial_population: Belirtilen popülasyon büyüklüğünde rastgele kromozomlar üretir. Yani genetik algoritmanın başlangıç jenerasyonu, tamamen rastgele dağıtılmış teslimatlardan oluşur.
- crossover: İki çözümü (ebeveyn kromozomlar) alıp bunlardan yeni bir **çocuk çözüm** üretir. Basit bir yaklaşım olarak her teslimat için rastgele ebeveynlerden miras alma yöntemi kullanılmıştır. Fonksiyon, iki ebeveynin tüm teslimatlarını bir birleşim kümesinde toplar, sonra bu teslimatları tek tek ele alıp yazı-tura şeklinde anne ya da babadan gelen atamayı çocuğa aktarır. Böylece çocuk kromozom, her iki ebeveynlerden karışık şekilde alınmış bir rota setine sahip olur. Bu yöntem teslimatları dağıtırken ebeveynlerin iyi yönlerinin kombinasyonunu yeni bireye aktarmayı hedefler.
- mutate: Tek bir kromozom üzerinde küçük rastgele değişiklik yapar (mutasyon). Burada gerçekleştirilen mutasyon tipi, rastgele seçilen bir teslimatı, farklı bir drone'a veya aynı drone'un rotasında farklı bir pozisyona taşımaktır. Bu işlem, çözümlerin çeşitliliğini artırarak yerel optimumlara takılma olasılığını azaltır.
- genetic_algorithm: Genetik algoritmanın ana döngüsünü gerçekleştiren fonksiyondur. Belirtilen popülasyon büyüklüğü kadar rastgele birey ile başlar ve belirlenen jenerasyon sayısı boyunca evrimleştirme yapar. Her jenerasyonda:
 - Tüm popülasyon bireylerinin fitness değeri hesaplanır ve bireyler en iyiden en kötüye sıralanır
 - **Elitizm** uygulanarak en iyi %10'luk kesim (en yüksek puanlı bireyler) doğrudan bir sonraki nesle kopyalanırGitHub. Örneğin popülasyon 20 ise en iyi 2 birey korunur.
 - Kalan yeni nesil bireyleri üretmek için crossover ve mutasyon kullanılır. Koddaki, yeni bir birey oluştururken önce rastgele %70 olasılıkla crossover

yapılır (aksi halde mevcut popülasyondan rastgele bir birey aynen alınır), ardından %20 olasılıkla üretilen bireye mutasyon uygulanırGitHub. Bu süreç yeni popülasyon boyutu istenen seviyeye ulaşana dek tekrarlanır.

- Yeni popülasyon bir sonraki jenerasyon olarak değerlendirilir. Eğer mevcut jenerasyonda geçmişe kıyasla daha iyi bir çözüm bulunursa, `best_solution` ve `best_fitness` güncellenir.

Bu döngü sonunda en iyi bulunan çözüm ve puanı döndürülürGitHub. Genetik algoritma, bu proje kapsamında **teslimat atama problemini çözmede merkezi rol oynamaktadır**. Teslimat sayısı büyüdükçe çözüm uzayı çok büyük olabileceğinden, genetik algoritma yaklaşımla kabul edilebilir bir çözüm aranmıştır.

V. KULLANILAN ÖNEMLİ ALGORİTMALAR VE İŞLEYİŞLERİ

Projenin bel kemiğini oluşturan iki ana algoritma vardır: **Genetik Algoritma (GA)** ve *A arama algoritması**. Ayrıca, no-fly zone kısıtlarını kontrol etmek için kullanılan **çizgi kesişimi algoritması** da önemli bir yardımcıdır.

Genetik Algoritma (GA): Yukarıda modül bazında detaylandırdığımız gibi, GA proje kapsamında **drone görev atama problemine** uygulanmıştır. Problem doğası gereği NP-zor kategoride olduğundan (bir tür kombinatoriyel optimizasyon), tüm olası atamaları deneyerek en iyiyi bulmak pratik değildir. Bunun yerine GA ile **yaklaşık iyi çözümler** aranır. İşleyiş şu şekildedir:

- İlk olarak, problem için uygun bir **kodlama (genom temsili)** tanımlanmıştır. Her olası çözüm, her drone için bir teslimat listesi olacak şekilde sözlük olarak temsil edilir. Böyle bir yapıda tüm teslimatlar bir drone'a atanmış olur. Bu genom yapısı üzerinde crossover ve mutasyon gibi genetik operatörlerin tanımlanması kolaylaşır.
- Başlangıç Popülasyonu:** GA, onlarca farklı rastgele çözümle başlar. `create_initial_population` fonksiyonu, her biri tamamen rastgele dağıtılmış teslimatlardan oluşan bir dizi kromozom yaratır. Bu popülasyonun amacı, arama uzayını geniş bir yelpazede tarayabilmektir.
- Değerlendirme (Fitness Hesabı):** Her bir birey (çözüm) fitness fonksiyonu ile değerlendirilir. Fitness fonksiyonu, bir çözümün ne kadar iyi olduğunun sayısal bir ölçüsüdür. Bu projede fitness, teslimat sayısı arttıkça yükselen, ancak ihlaller ve enerji tüketimi arttıkça düşen bir puanlama sistemidirGitHub. Hesaplama sürecinde

her drone'un rotası tek tek incelenir; kapasite, menzil, no-fly zone ve zaman ihlali durumları tespit edilip cezalar uygulanır. Örneğin aşağıdaki kod, bir drone için kapasite, batarya ve no-fly zone kontrollerini göstermektedir:

GitHub Bu kod parçasında, eğer teslimatın ağırlığı drone'un **maksimum taşıma kapasitesinden fazlaysa** veya teslimat noktasına **mesafe drone'un kalan bataryasını aşıyorsa**, ilgili teslimat *yapılmıyor* sayılıp `continue` ile döngünün sonraki teslimatına geçiliyor (ve her iki durumda da `total_violations` artırılıyor). Ardından, drone ile teslimat noktası arasındaki hat boyunca bir **no-fly zone engeli** olup olmadığı `edge_crosses_noflyzones` fonksiyonuyla kontrol ediliyor. Eğer bu hat bir yasaklı bölgeyi kesiyorsa, bu da bir ihlal olarak sayılıyor (`total_violations` artıyor) ve drone yine de o noktaya giderken bataryasını harcıyor fakat zaman ilerletilmiyor (ihlal cezası olarak). Görüldüğü gibi, fitness hesabı drone rotalarını **detaylı bir şekilde simüle ederek** bu çözümün ne kadar geçerli olduğunu ölçüyor.

- Seçim ve Üreme:** Her nesilde, yüksek puan alan çözümlerin hayatta kalma ve yeni nesle gen aktarım olasılığı daha yüksektir. Kodda bu, elitizm ve rastgele seçim karışımıyla sağlanmıştır. En iyi bireylerin küçük bir yüzdesi doğrudan kopyalanırken, kalan yeni bireyler mevcut popülasyondan seçilenlerin crossover ve mutasyona uğramasıyla üretilirGitHub. Crossover, iki çözümün bilgilerini karıştırarak potansiyel olarak daha iyi bir çözüm ortaya çıkarabilir. Mutasyon ise rastgele küçük değişikliklerle çeşitliliği artırır.
- Döngü ve Sonlanma:** Belirtilen jenerasyon sayısı kadar bu süreç devam eder veya istenirse belirli bir fitness eşiği sağlandığında da durdurulabilir (mevcut implementasyonda jenerasyon sayısı sabit tutulmuş). Sonunda en yüksek fitness'lı çözüm sonuç olarak seçilir. Bu çözüm, her bir drone'un hangi teslimatları hangi sırayla yapacağını gösterir.

GA'nın avantajı, çok büyük ve kompleks arama uzaylarında çalışabilmesidir. Bu projede de 40-50 teslimatlık senaryolarda dahi makul sürede iyi bir çözüme ulaşmak hedeflenmiştir. Ancak GA doğal olarak **kesin optimum garantisi vermez**, bu nedenle farklı denemelerde farklı çözümler üretilip en iyisini almak gerekebilir. Kodda rastgelelik kullanıldığı için (ör. başlangıç popülasyonu ve mutasyonlar), aynı senaryo birden fazla kez çalıştırılarak ortalama bir performans görülebilir.

A Algoritması:* Projede A* (A-star) algoritması, olası rotalarda no-fly zone'lardan kaçınarak **en kısa yol bulmak** amacıyla implemente edilmiştir. Her ne kadar genetik algoritma drone'ların hangi teslimatları alacağını belirlese de, iki nokta arasındaki gerçek rotanın bulunması ayrı bir problemdir (çünkü no-fly zone etrafından dolanmak gerekebilir). A* tam bu işi yapmak için uygundur: `build_graph` fonksiyonuyla oluşturulan düğüm-kenar grafında, bir başlangıç noktası (drone'un konumu) ile bir teslimat noktası arasında en düşük maliyetli yolu hesaplar. Maliyet fonksiyonu basitçe mesafe, paket ağırlığı ve öncelik

bileşenlerinden oluşur; eğer bir kenar no-fly zone içinden geçiyorsa bu kenara büyük bir **ceza maliyeti** eklenerek A*’ın onu tercih etmemesi sağlanır. A* aramasının kendisi, tipik bir öncelikli kuyruk tabanlı en kısa yol aramasıdır – her bir düğüm için $f_score = g_score + heuristic$ değerini kullanır ve hedefe ulaştığında durur GitHub GitHub. Bu projede heuristic olarak **düz mesafe** kullanılmıştır (yani uçabileceği en iyi senaryodaki mesafe). A* algoritmasının doğru çalışabilmesi için build_graph fonksiyonunun ürettiği graf önemlidir; bu graf, drone’ların gidemeyeceği yolları (kapasite/batarya yetersizliği) hiç eklemeyerek A*’ın arama uzayını daraltır GitHub. Sonuç olarak, A* eğer bir drone ile teslimat noktası arasında no-fly zone’u dolanarak bir yol bulmak mümkünse, onu bulacaktır.

Mevcut kodda A* fonksiyonu çağrılmamış olsa da, **ileriye dönük iyileştirme potansiyeli** taşır: Örneğin genetik algoritma bir drone’a belirli bir teslimat sırası atadıktan sonra, her bir ardışık teslimat çifti arasında A* ile tam uçuş rotası çıkarılabilir. Bu sayede eğer düz gitmek imkansızsa dolambaçlı rota hesaplanıp drone’un harcayacağı gerçek mesafe bulunabilir. Şu an ise yaklaşım, eğer düz hatta bir engel varsa bunu ihlal sayıp cezalandırmak biçimindedir – yani drone’un dolanacağı varsayılmamıştır. İleride A* bütünleşirse, no-fly zone ihlallerine ceza vermek yerine, drone rotasını değiştirerek ceza almadan hedefe ulaşması simüle edilebilir.

No-Fly Zone Kesişim Algoritması: Projede ufak ama önemli bir algoritma parçası da no-fly zone kesişim kontrolüdür. Bu, her bir drone uçuş çizgisinin (iki nokta arasındaki düz hat) çokgen şeklindeki bir no-fly bölgesiyle kesişip kesişmediğini hesaplar. Kodda bu amaçla klasik bir geometrik algoritma uygulanmıştır: Bir doğru parçasının bir çokgeni kesip kesmediği, doğru parçasıyla çokgenin her bir kenarının kesişim kontrolü yapılarak anlaşılabilir. segments_intersect fonksiyonu, iki doğru parçasının kesişimini basit bir **CCW (counter-clockwise)** yön kontrolüyle belirler. edge_crosses_noflyzones fonksiyonu ise drone’un mevcut konumu ile teslimat noktası arasındaki hattı alır ve tüm no-fly zone’ların kenarları ile test eder GitHub. Eğer bir tane bile kesişim bulunursa True döner (yani bu hat bir yasaklı bölgeden geçiyor demektir). Bu sonuç, hem fitness hesaplamada hem de graf oluştururken kullanılır. Bu algoritma parçası, matematiksel olarak nispeten basit olsa da, proje açısından kritik bir işleve sahiptir: No-fly zone engellerini pratik bir şekilde tespit etmemizi sağlar. Bu sayede dronelerin rotalarını belirlerken haritanın yasak bölgelerini hesaba katabiliyoruz.

Yukarıda açıklanan genetik algoritma çıktısının bir örneği. Şekilde üç adet drone (mavi kare noktalar) ve on adet teslimat noktası (yeşil daireler) görülmektedir. Kırmızı kesik çizgiler iki adet no-fly zone bölgesinin sınırlarını gösterir. Her bir drone, kendine atanan teslimat noktalarını sırasıyla ziyaret etmektedir; örneğin turuncu çizgi Drone 1’in rotasını belirtir. Bu görsel, projenin algoritmalarının ürettiği çözümü harita üzerinde somutlaştırmaktadır.

VI. TEST YAPISI

Projenin depo içeriğinde **birim testler veya otomatik testlere dair özel bir yapı bulunmamaktadır**. Yani

unitest benzeri çerçeveler kullanılmamış, test dosyaları yazılmamıştır. Bunun yerine, test etme işlemi daha çok **senaryo çalıştırma ve sonuç gözlemlene** şeklinde gerçekleştirilmiştir. main.py dosyasında farklı senaryolar arka arkaya çalıştırılarak sonuçların konsolda çıktısı alınmaktadır GitHub GitHub. Örneğin temel, büyük, çok büyük senaryolar art arda koşturularak her biri için bulunan sonuçlar (tamamlanan teslimat yüzdesi, fitness puanı, ihlal sayıları vb.) ekrana basılıyor. Geliştirici muhtemelen bu çıktılara bakarak algoritmanın performansını ve doğruluğunu değerlendirmiştir.

Rastgele senaryo üretme özelliği de bir anlamda test amacı taşır: Sistem beklenmedik dağılımlarda da çalışıp çalışmadığı, herhangi bir hata verip vermediği gözlemlenebilir. Örneğin, 5 drone ve 20 teslimatlı bir senaryo ile 10 drone 40 teslimatlı bir senaryo, kodun ölçeklenebilirliği ve stabilitesi için farklı test ortamları sağlar. Kod yapısı, farklı büyüklükteki girişlere uyum sağlayabilecek şekilde yazıldığından (ör. popülasyon boyutu teslimat sayısına göre ayarlanıyor), bu esneklik çeşitli testlerde denenmiş olmalıdır.

Bir diğer test yöntemi olarak, **manuel gözlem ve görselleştirme** kullanılmıştır. Sonuçların harita üzerinde çizdirilmesi, geliştiriciye çözümün mantığı hakkında sezgisel bir geri bildirim verir. Örneğin bir drone’un rotasının no-fly zone’u kestiği görselden anlaşılırsa, buna karşılık konsolda o teslimat için kural ihlali raporlanıp raporlanmadığı kontrol edilebilir. Bu tür çapraz kontroller, sistemin kural setini doğru uyguladığını test etmek için kullanışlıdır.

Özetle, test süreci büyük ölçüde **senaryo tabanlı deneyler** üzerinden yürütülmüştür. Küçük ölçekli senaryolarla başlanıp sonuçlar doğrulandıktan sonra daha büyük senaryolara geçilmiş olması muhtemeldir. Her bir senaryo sonunda konsolda yazılan değerler (başarı oranı, ihlal sayıları vb.) projenin beklenen davranışları gösterip göstermediğini anlamak için değerlendirilmiştir. Resmi bir birim/entegrasyon testi bulunmasa da, projenin kendi içindeki bu **doğrulama mekanizmaları** ihtiyaçları karşılamıştır.

VII. KOD KALİTESİ

Projedeki kod kalitesi genel olarak **temiz ve anlaşılır** bir düzeydedir. Kod yapısı modüler olarak ayrılmış, uzun fonksiyonlardan kaçınılmış ve her dosya tek bir sorumluluk üstlenecek şekilde tasarlanmıştır. Değişken ve fonksiyon isimleri, yaptıkları işi açıklayıcı niteliktedir (örn. edge_crosses_noflyzones, generate_random_deliveries, fitness gibi isimler, fonksiyonun amacını net olarak belli etmektedir). Python’un **tip ipuçları** (type hints) kullanılmış olması, özellikle fonksiyon imzalarında ve veri yapılarında kodun anlaşılmasını kolaylaştırır – hangi fonksiyonun ne tipte parametre aldığı ve ne döndürdüğü hemen görülebilir.

Kod içinde yer yer açıklayıcı **yorum satırları ve dokümantasyon satırları** bulunur. Örneğin bir fonksiyonun başında üç tırnak içinde Türkçe olarak ne yaptığı

anlatılmıştırGitHub. A* fonksiyonunun, fitness fonksiyonunun ve mutata gibi yardımcıların docstring'leri mevcuttur. Bu dokümantasyonların Türkçe yazılmış olması, projeyi Türkçe bilen geliştiriciler için rahat okunur hale getirirken, evrensel geliştirici kitlesi için bir miktar kısıtlama getirebilir. Ancak projenin muhtemelen eğitim/ödev ya da yerlere yönelik bir çalışma olduğu düşünüldüğünde, Türkçe dokümantasyon anlaşılır ve yeterlidir. Bunun yanında, kod içerisinde önemli adımlar da yorumlarla belirtilmiştir (mesela # Kapasite kontrolü, # No-fly zone kontrolü gibi açıklamalarGitHub). Bu, geliştirici deneyimini iyileştiren bir unsurdur; koda bakan biri, ilgili bloğun neyi kontrol ettiğini hemen anlayabilir.

Dış dokümantasyon konusunda ise proje zayıftır. Depoda bulunan README.md dosyası neredeyse boş durumdadır – sadece projenin adını içeren tek satırlık bir başlık vardır ve o da muhtemelen yanlışlıkla “dron2” şeklinde kalmıştırgithub.com. Yani proje hakkında dışarıya bilgi veren bir açıklama, kullanım talimatı, ön koşullar vs. bu README’de yer almamaktadır. Bu durumda, projeyi çalıştırmak isteyen bir geliştirici, doğrudan kodu incelemek zorunda kalacaktır. Neyse ki kodun yapısı sezgiseldir: main.py incelendiğinde, programın nasıl çalıştığı, hangi dosyaları okuduğu anlaşılabilir. Yine de, örneğin *"Proje ne yapar, nasıl çalıştırılır"* gibi temel bilgilerin README’de olmaması, proje dışı kullanıcılar için dezavantajdır.

Kod kalitesi açısından bir diğer nokta, **hata durumu kontrolü**dür. Projede istisna yakalama (try/except) blokları neredeyse hiç yoktur. Bu, girdi dosyalarının doğru formatta olduğunu varsaymak gibi yaklaşımların benimsendiğini gösterir. Örneğin data_loader.py dosyası dosya formatının uygun olduğunu kabul ederek satırları parçalayıp parse ediyor; hatalı bir giriş durumunda (örneğin eksik sütun) program muhtemelen hata fırlatacaktır. Bu tip durumların ele alınmamış olması, kodu daha basit tutma tercihindən kaynaklanmış olabilir, ancak üretim kalitesinde bir projede iyileştirilmesi gereken bir yön olarak not edilebilir.

Geliştirici deneyimi açısından değerlendirirsek: Proje küçük ve yönetilebilir boyutta olduğundan, yeni bir geliştiricinin kodu anlaması fazla zaman almayacaktır. Modüllerin adları ve fonksiyonların imzaları kendini açıklayıcı olduğu için, temel algoritma bilgisine sahip birinin projeyi çözmesi kolaydır. Genetik algoritma gibi karmaşık bir sürecin dahi adım adım okunabilir olması, kodun temiz yazılmış olmasındandır. Örneğin genetic_algorithm fonksiyonunun yapısı, gen0'dan genN'e kadar evrimi düz bir şekilde kodlamış ve en karmaşık kısmı olan “yeni nesil üretme” bölümünü bile anlaşılır kılmıştır. Bu, geliştiricinin algoritmayı net kavrayıp kodladığını gösterir.